

Dido's BreakOut - Relazione

A.A. 2025/2026

Achille Montefiori · Fabio Cavina · Lorenzo Di Domenico · Riccardo Frega

1. Analisi

1.1 Descrizione e requisiti

Il software realizzato è una reinterpretazione 2D in stile arcade del celebre videogioco *Atari Breakout*, pensata per un singolo giocatore su un dispositivo desktop. Il giocatore controlla una barra orizzontale (il *paddle*) che può muovere a destra e a sinistra per far rimbalzare verso l'alto una pallina; la pallina, muovendosi sul campo di gioco, colpisce e distrugge i mattoni di una griglia. L'obiettivo è distruggere quanti più mattoni possibile, accumulando punteggio, evitando che la pallina cada oltre il bordo inferiore.

A differenza del Breakout classico, suddiviso in livelli discreti, questa versione è a **livello infinito**: la griglia di mattoni scorre con continuità verso il basso, viene rigenerata via via in cima al campo, e la sua velocità di discesa cresce progressivamente, rendendo la partita sempre più difficile. La partita non ha quindi una condizione di vittoria, ma soltanto di sconfitta: si termina quando il giocatore esaurisce le vite a disposizione (la pallina cade oltre il bordo inferiore) oppure quando una riga di mattoni raggiunge l'altezza del paddle.

Il campo di gioco contiene diverse tipologie di mattoni, con comportamenti differenti: mattoni a resistenza singola (distrutti in un colpo), mattoni a resistenza multipla (che richiedono più colpi), mattoni indistruttibili, mattoni speciali che al momento della distruzione rilasciano una capsula, e mattoni esplosivi che, se colpiti, danneggiano anche i mattoni adiacenti. La capsula, una volta rilasciata, cade verso il basso e, se intercettata dal paddle, attiva un effetto temporaneo che modifica il gameplay: tali effetti possono essere vantaggiosi (*power-up*, ad esempio paddle ingrandito, blocchi congelati e punti raddoppiati) o svantaggiosi (*malus*, ad esempio paddle rimpicciolito, pallina più veloce, punti dimezzati).

Al termine della partita il punteggio raggiunto può entrare in una classifica persistente dei migliori risultati, mantenuta tra una sessione e l'altra. L'applicazione presenta inoltre un menù principale per l'avvio della partita e una schermata di fine partita che mostra il punteggio finale e consente di rigiocare o uscire.

Requisiti funzionali

- **Controllo del paddle:** il giocatore deve poter muovere il paddle orizzontalmente a destra e a sinistra, entro i limiti del campo di gioco.
- **Movimento e rimbalzo della pallina:** la pallina si muove di moto rettilineo uniforme e rimbalza contro le pareti laterali e superiore, contro i mattoni e contro il paddle; il

rimbalzo sul paddle deve variare l'angolo della traiettoria in funzione del punto di impatto, per dare al giocatore il controllo della direzione.

- **Campo di gioco e griglia:** il sistema deve gestire un campo 2D delimitato e popolato da una griglia di mattoni.
- **Generazione continua del livello:** le righe di mattoni devono essere generate con continuità e scorrere verso il basso, con velocità crescente nel corso della partita, senza interruzioni di livello.
- **Tipologie di mattoni:** il sistema deve gestire mattoni a resistenza singola, a resistenza multipla, indistruttibili, speciali (rilascio di capsule) ed esplosivi.
- **Rilevamento e gestione delle collisioni:** il sistema deve rilevare gli urti della pallina con pareti, paddle e mattoni, applicando il corretto rimbalzo e, nel caso dei mattoni, il danno conseguente con eventuale rimozione.
- **Sistema di punteggio:** il punteggio deve incrementarsi a ogni mattone colpito o distrutto, ed essere influenzato dagli effetti attivi (ad esempio raddoppio o dimezzamento dei punti).
- **Gestione delle vite e fine partita:** il sistema deve decrementare le vite alla caduta della pallina e dichiarare il game over all'esaurimento delle vite o quando una riga di mattoni raggiunge il paddle.
- **Power-up e malus:** il sistema deve gestire la caduta delle capsule, la loro raccolta da parte del paddle e l'attivazione di effetti temporanei, vantaggiosi o svantaggiosi.
- **Classifica persistente:** il sistema deve mantenere e mostrare una classifica dei migliori punteggi, conservata tra sessioni di gioco distinte.
- **Interfaccia e navigazione:** l'applicazione deve fornire un menu principale, una visualizzazione a schermo del punteggio corrente e delle vite rimanenti, e una schermata di fine partita con possibilità di rigiocare.

Requisiti non funzionali

- **Reattività e fluidità:** l'interfaccia deve aggiornarsi in tempo reale e con fluidità, garantendo un'esperienza di gioco continua.
- **Robustezza del sistema di collisioni:** il rilevamento delle collisioni deve essere affidabile anche ad alta velocità della pallina, evitando il fenomeno del *tunneling* (attraversamento di un mattone senza registrare l'urto) e i loop infiniti di rimbalzo tra pareti o mattoni indistruttibili.
- **Stabilità della generazione infinita:** la generazione continua delle righe deve mantenere prestazioni stabili nel tempo ed evitare la creazione di barriere orizzontali invalicabili composte da mattoni indistruttibili.
- **Portabilità:** il software deve essere eseguibile sulle principali piattaforme desktop che supportino l'ambiente di esecuzione scelto.

1.2 Modello del dominio

Il sistema ruota attorno al concetto di **Partita**, che rappresenta una sessione di gioco e ne aggrega gli elementi costitutivi: la **Pallina**, il **Paddle** e il **Livello**. La Partita mantiene inoltre lo stato di avanzamento del giocatore, ovvero il **Punteggio** corrente e il numero di **Vite** rimanenti.

La **Pallina** è l'entità centrale: si muove sul campo e interagisce con il **Paddle** (rimbalzo con angolo dipendente dal punto di impatto), con le pareti del campo (rimbalzo diretto) e con i **Mattoni** (collisione con conseguente danno). Quando la pallina supera il bordo inferiore, il giocatore perde una Vita.

Il **Livello** rappresenta la griglia di gioco: aggrega l'insieme dei Mattoni attivi, li fa scorrere verso il basso e ne genera di nuovi con continuità. Il **Mattone** è l'elemento colpibile della griglia e si specializza in più varianti in base al comportamento: il **Mattone resistente** richiede più colpi, il **Mattone indistruttibile** non può essere distrutto, il **Mattone esplosivo** propaga il danno ai mattoni adiacenti, e il **Mattone speciale**, al momento della distruzione, rilascia una **Capsula**.

La **Capsula** cade verso il basso e, se raccolta dal Paddle, attiva un **Effetto** temporaneo che altera lo stato della Partita: può modificare il comportamento della Pallina (ad esempio la velocità), del Paddle (ad esempio la larghezza) o del Punteggio (ad esempio raddoppiandolo o dimezzandolo), per una durata limitata.

Infine, la **Classifica** rappresenta l'elenco persistente dei migliori punteggi: al termine di una partita, il punteggio raggiunto può esservi registrato.

Lo schema UML del modello del dominio è riportato in Figura 1.1.

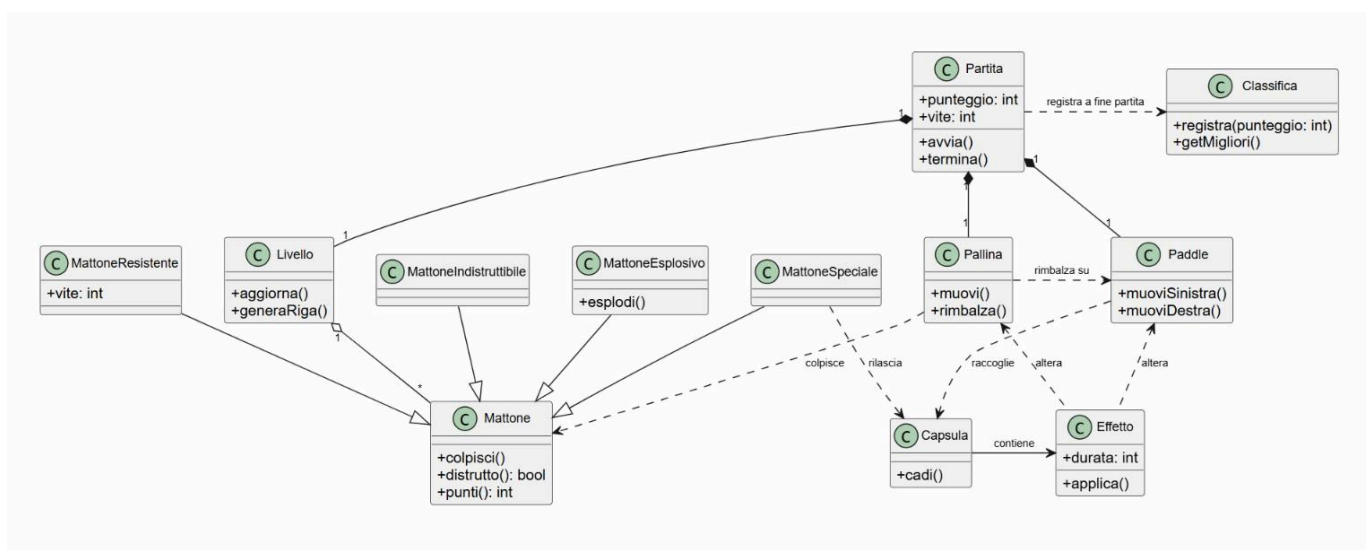


Figura 1.1: Schema UML del modello del dominio.

2. Design

2.1 Architettura

L'architettura di Breakout adotta il pattern architetturale Model-View-Controller (MVC), scelta naturale per un gioco arcade in tempo reale governato da un game loop: il disaccoppiamento netto tra logica di dominio, rappresentazione visiva e gestione dell'input permette di far evolvere lo stato del gioco a ogni fotogramma mantenendo le tre componenti indipendenti. Il sistema è orchestrato da un game loop centrale, realizzato con un `javax.swing.Timer` con intervallo di 16 ms (~60 FPS) gestito dal Controller, che a ogni tick coordina l'aggiornamento del Model e il ridisegno della View.

I vertici del triangolo MVC sono individuati dalle seguenti interfacce di ingresso:

- *Model*: il nucleo della logica risiede in un insieme di interfacce di dominio — `Ball`, `Paddle`, `LevelManager`, `CollisionManager` — affiancate dai gestori `LivesManager` e `PowerUpManager`, oltre a `Leaderboard` per la persistenza dei punteggi. Il Model è completamente agnostico rispetto all'interfaccia utente: nessuna sua classe importa `Swing` né possiede riferimenti a View o Controller.
- *View*: l'interfaccia `GameMap` (con l'implementazione `GameMapImpl`, una finestra `JFrame`) funge da coordinatore dell'interfaccia, contenendo i sotto-pannelli specializzati: `MainPanel` per il rendering del campo di gioco, `LeftPanel` e `RightPanel` per la visualizzazione di punteggio, vite ed effetti attivi (HUD), e le schermate `MenuView` e `GameOverView`. La View è passiva: non prende decisioni di gioco, ma si limita a renderizzare lo stato che legge dal Model e a notificare il Controller degli input.
- *Controller*: `GameController`, che implementa `KeyListener`, agisce da regista: contiene il game loop, riceve gli input da tastiera, interroga e aggiorna il Model e comanda il ridisegno della View. La creazione e il cablaggio di tutti i componenti è delegata alla classe `App`, che tramite il factory method `startGame()` istanzia Model, View e Controller e li collega tra loro.

Interazione e flusso dati

All'avvio, `App.startGame()` crea le entità di dominio, la finestra di gioco e il controller, iniettando in quest'ultimo i riferimenti alle interfacce del Model e della View. A ogni iterazione del game loop, `GameController.update()`:

1. legge lo stato degli input (tasti premuti) e muove il paddle, fa avanzare la pallina e aggiorna il livello;
2. invoca `CollisionManager` per rilevare e applicare le collisioni tra pallina, paddle, pareti e mattoni;
3. interroga `PowerUpManager` e `LivesManager` per aggiornare l'HUD (effetti attivi e vite) sui pannelli laterali;
4. verifica le condizioni di fine partita - `livesManager.isGameOver()` oppure il superamento della soglia da parte di una riga di mattoni (`levelManager.hasBricksReachedThreshold()`) - e infine richiede il `repaint()` della View.

Per garantire il disaccoppiamento, il Model non comunica mai direttamente con la View: è il Controller a fare da tramite, e i pannelli leggono lo stato di dominio (posizione di pallina e paddle, mattoni attivi, effetti) attraverso le interfacce del Model nel metodo paintComponent, senza poterlo alterare.

Un elemento architetturale rilevante è la dependency injection dei gestori: CollisionManager non gestisce più in proprio vite e power-up, ma riceve nel costruttore LivesManager e PowerUpManager, delegando loro la rispettiva logica. Questa separazione confina CollisionManager al solo compito di rilevare e applicare le collisioni, e permette al Controller di interrogare direttamente i gestori per aggiornare l'HUD senza attraversare la logica di collisione, migliorando coesione e testabilità.

Sostituibilità della View

Grazie alla separazione tramite interfacce, la rappresentazione visiva è in larga parte indipendente: i pannelli leggono il Model solo attraverso le sue interfacce e il Controller dialoga con il dominio senza conoscere i dettagli grafici, per cui sostituire il renderer del campo o l'intero insieme di pannelli non impatta Model né logica di controllo. Va segnalata onestamente una limitazione attuale: GameMapImpl è referenziata in modo concreto all'interno di GameController anziché tramite l'interfaccia GameMap, per cui una sostituzione in blocco della finestra principale richiederebbe una piccola modifica al Controller. È un punto di miglioramento noto, che non compromette il disaccoppiamento dei layer ma ne riduce la purezza.

Lo schema generale dell'architettura è riportato in Figura 2.1.

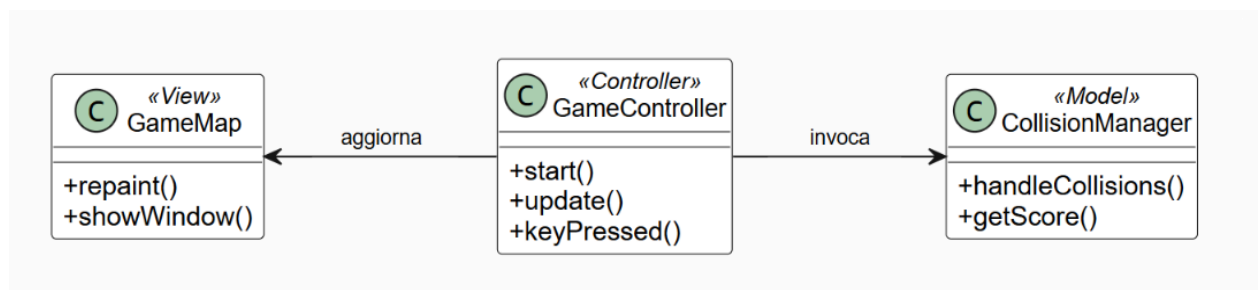


Figura 2.1: Schema UML dell'architettura MVC.

2.2 Design dettagliato

2.2.1 Montefiori

Risoluzione e logica dei blocchi e del livello infinito

Riguardo alla gestione del livello infinito e della creazione dei blocchi ci sono stati ostacoli da risolvere o gestire, i principali sono stati i seguenti:

- Problema della creazione di blocchi

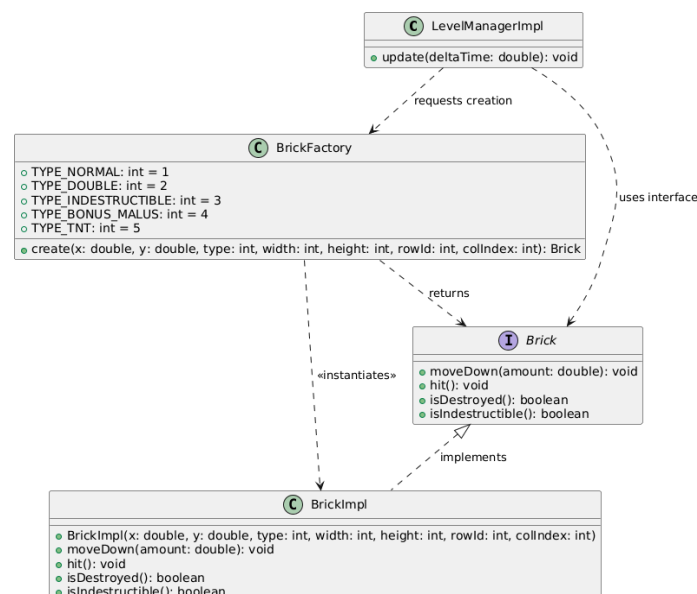
All'interno del gioco sono presenti diverse tipologie di blocchi (normali, resistenti, indistruttibili, bonus/malus e TNT) ed ogni blocco ha proprietà fisiche differenti, come ad esempio la vita iniziale o l'indistruttibilità. Nel caso si istanzia direttamente le classi all'interno della classe logica che gestisce il livello, creerebbe un accoppiamento troppo rigido creando non pochi problemi riguardo all'eventuale aggiunta di nuovi tipi di blocchi.

Soluzione e considerazioni alternative

Per garantire il disaccoppiamento tra i blocchi e la gestione del livello, si è deciso di nascondere l'implementazione concreta di brickimpl dietro l'interfaccia pubblica brick situato nel pacchetto API, così la creazione dei blocchi è stata interamente gestita all'interno della classe BrickFactory (Metodo Factory). I tipi di blocchi sono stati codificati come costanti statiche intere all'interno della stessa factory, ed il metodo statico create riceve la costante numerica che corrisponde alla classe concreta di brickimpl che restituisce al client l'interfaccia generica brick.

Un'alternativa che è stata considerata riguardava il creare una classe concreta differente per ogni tipo di mattoncino, come ad esempio NormalBrick, ExplosiveBrick ecc... , solo che la soluzione è stata scartata poichè principalmente i blocchi si differenziano riguardo attributi che sono numerici come la vita iniziale o indistruttibilità, inoltre mappare un'unica classe BrickImpl permette di mappare direttamente l'output del generatore casuale alla factory del blocco, evitando una gerarchia di classi aggiuntive.

Il ruolo di Client è svolto dal LevelManagerImpl, la Factory è implementata dalla classe BrickFactory e il Prodotto astratto restituito è rappresentato dall'interfaccia Brick. La Factory centralizza la logica di instradamento del tipo di blocco sollevando il client dalla necessità di conoscere i dettagli costruttivi di BrickImpl.

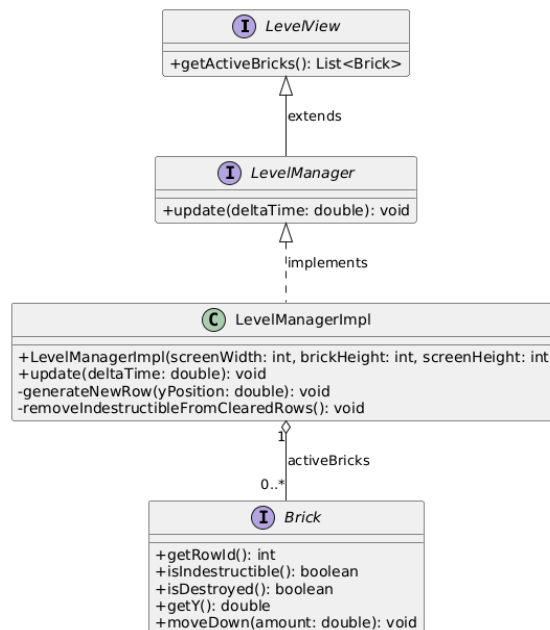


- gestione dinamica del livello e pulizia automatica delle righe

Per far sì che il livello continui a scorrere in automatico dando la logica del livello infinito, bisogna gestire la generazione dinamica delle righe fuori dallo schermo e di eliminare le righe distrutte, inoltre nel gioco si prevede che i mattoni indistruttibili dato che non devono fare perdere il giocatore, quando vengono distrutti tutti i blocchi bisogna anche implementare un modo per rimuoverli in modo automatico dalla schermata.

Soluzione e alternative:

Nel `LevelManagerImpl` si è scelto di utilizzare al suo interno una lista di tipo `List<Brick>` contenente tutti i blocchi attivi. Durante l'aggiornamento di ogni frame con metodo `update`, le posizioni dei blocchi vengono traslate verso il basso e viene eseguita la rimozione degli elementi distrutti o usciti dallo schermo, mentre per la pulizia dei blocchi indistruttibili è stato implementato il metodo privato `removeIndestructibleFromClearedRows()`, che raggruppa i blocchi attivi per il loro indice di riga (`rowId`) e se per una determinata riga non viene rilevato alcun blocco distruttibile allora l'intera riga di blocchi rimanenti che sarà composta solo da blocchi indistruttibili verrà rimossa dalla lista attiva.



- Aspetto: Ridimensionamento delle entità (Resize)

Nel momento in cui la finestra grafica dell'applicazione viene ridimensionata dall'utente, le posizioni e le dimensioni dei blocchi attivi devono adattarsi dinamicamente per mantenere la consistenza del livello di gioco e la spaziatura originale tra le righe di blocchi.

Soluzione proposta ed alternative considerate:

Il metodo `updateDimensions` calcola un fattore di scala proporzionale ($scale = \frac{newWidth}{screenWidth}$) basato sul cambiamento della coordinata orizzontale. Questo coefficiente viene utilizzato per ricalcolare uniformemente la spaziatura verticale originale del livello (`rowSpacing` e `distanceSinceLastRow`) e viene applicato a ciascun mattoncino attivo per ridimensionarne sia la posizione nel gioco (X, Y) sia la sua forma quadrata (larghezza e altezza).

2.2.2 Di Domenico

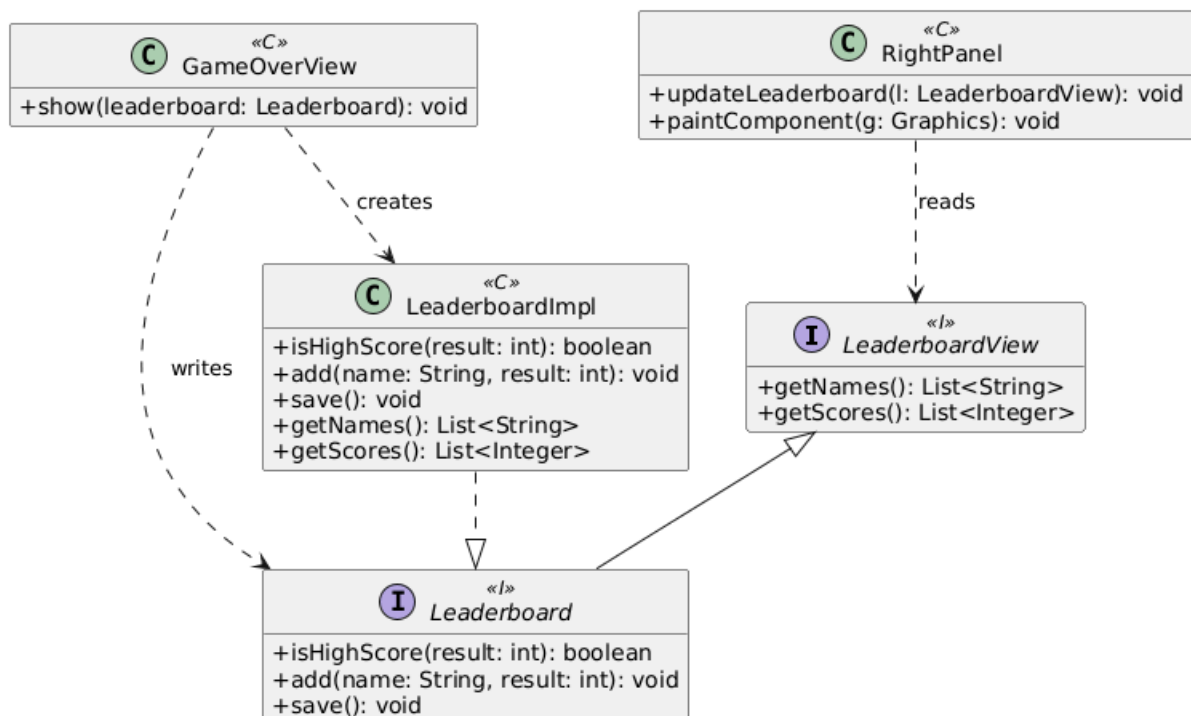
Problemi riguardo la leaderboard, la generazione dei bonus e malus e le condizioni di fine gioco

- **Gestione e persistenza della classifica dei punteggi**

Problema: Il gioco deve mantenere una classifica dei migliori punteggi. Questo comporta più necessità contemporaneamente: stabilire se un punteggio appena ottenuto è abbastanza alto da entrare in classifica, mantenere le voci ordinate e limitate a un massimo di dieci e leggere e scrivere questi dati su una memoria. La classifica prevede due punti distinti dell'applicazione, ovvero la sua visualizzazione a schermo e l'inserimento di un nuovo record al termine della partita.

- **Soluzione e considerazioni alternative:**

L'interfaccia Leaderboard comprende i metodi che servono per gestire la classifica che poi verrà salvata sul file di testo Leaderboard.txt: il metodo isHighScore ci dice se un punteggio è abbastanza alto da entrarvi, il metodo add, che inserisce una nuova voce mantenendo l'ordinamento e il limite di dieci posizioni e il metodo save scrive l'intera classifica corrente nel file di testo, sovrascrivendo il contenuto precedente. Da questa interfaccia si separa una mini-interfaccia LeaderboardView, che prevede solo i due getter di lettura (getNames e getScores). Dato che Leaderboard estende LeaderboardView, un oggetto della classifica può essere visto in due modi : come Leaderboard, in questo caso è possibile aggiungere voci e salvare su file, oppure come LeaderboardView, in questo caso si può solo consultare. Questa separazione è stata pensata per la view (nel RightPanel): il pannello che disegna la classifica a schermo riceve un riferimento di tipo LeaderboardView, in modo che dal suo punto di vista la classifica risulti di sola lettura e non sia modificabile neanche per sbaglio. La scrittura di un nuovo punteggio in classifica avviene nel GameOverView, dove, se il punteggio è idoneo ad entrare in classifica, viene richiesto di inserire il "nome" (3 lettere come nei vecchi giochi arcade) associato al suo punteggio.



- **Aspetto: gestione dei power up (bonus/malus)**

Problema: Nel gioco esistono sette tipi diversi di power-up (vita extra, paddle corto, punti doppi, paddle largo, blocchi congelati, punti dimezzati, palla veloce). Quando un blocco bonus viene distrutto deve nascere una capsula di tipo casuale che cade verso il paddle, che può essere un bonus (verde), o un malus (viola).

Soluzioni e considerazioni alternative:

Si è scelto di rappresentare le capsule con la classe `PowerUpImpl`. Qui dentro viene gestita anche la logica della caduta e la verifica di uscita dallo schermo, mentre la decisione di cosa fare quando la capsula viene raccolta è demandata al `PowerUpManagerImpl`, che riceve la capsula, ne legge il tipo e applica l'effetto corrispondente attraverso uno switch sui sette tipi di bonus/malus differenti, la scelta del potere è totalmente randomica e tutti i poteri hanno le stesse probabilità di uscire.

➤ **Gestione della durata dei poteri**

Ogni potere ha una durata di 8 secondi e una propria icona dedicata visibile a schermo per tutta la durata dell'effetto, nel momento in cui l'effetto preso sia la vita extra, l'icona della vita sarà visibile a schermo solo per pochi secondi(2s).

Si è scelto di misurare la durata degli effetti contando il numero di frame del gioco, per ogni effetto attivo, il manager mantiene un contatore che parte da un valore fisso al momento della raccolta della capsula e viene decrementato di uno ad ogni chiamata di `updateTimer`, eseguita una volta per frame dal game loop. Quando un contatore raggiunge zero, l'effetto viene annullato. È stata scelta questa configurazione rispetto all'utilizzo dei timer che contavano gli effettivi secondi di durata del potere perché così quando si mette in pausa il gioco, la durata del potere si fermerà con il gioco e riprenderà con il potere che durerà quanto tempo sarà rimasto fino allo scadere.

➤ **Gestione della vita extra**

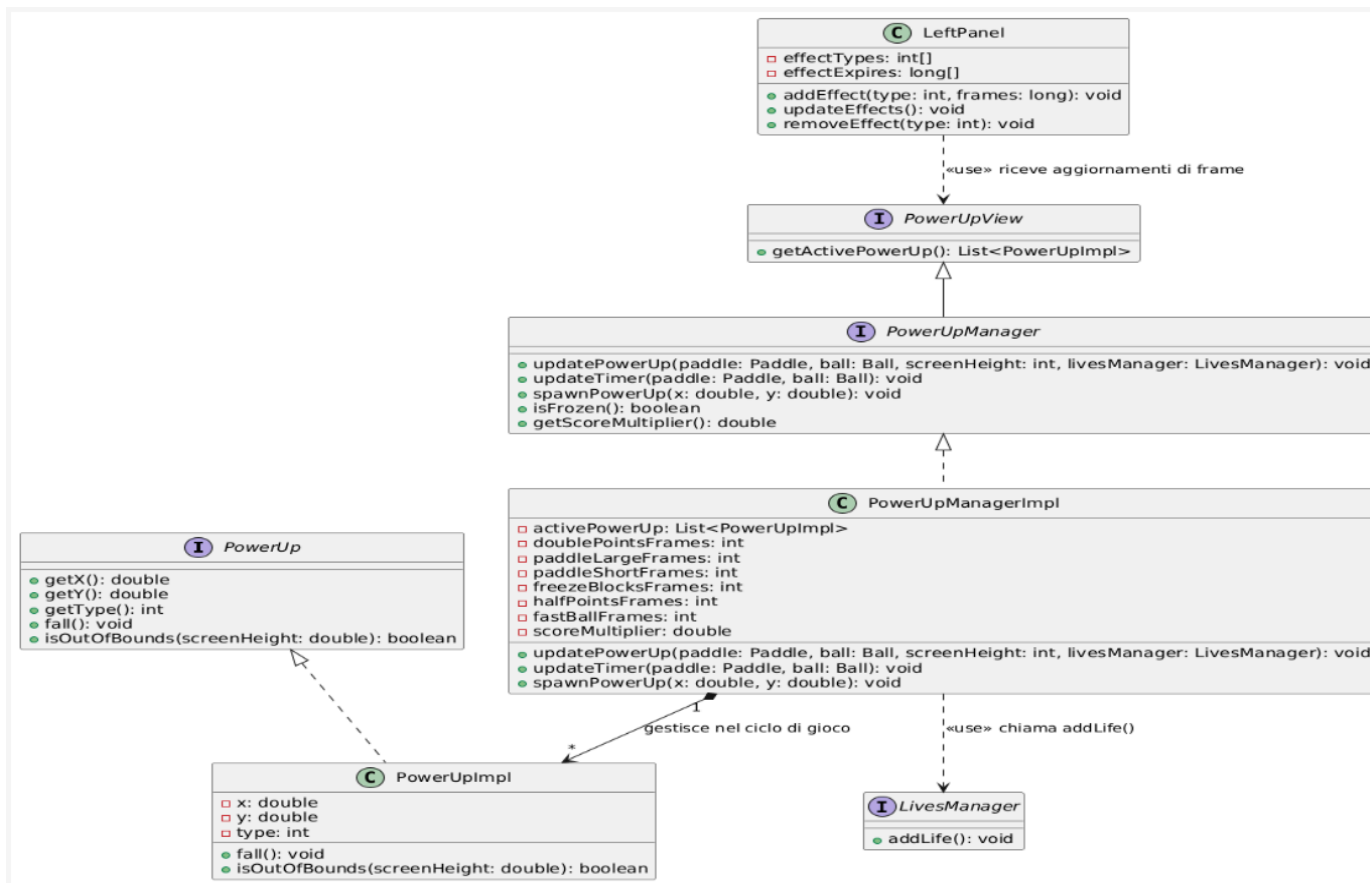
Come accennato prima la raccolta del potere della vita extra non ha una durata in quanto è un'azione istantanea.

Quando il giocatore raccoglie la capsula della vita extra, il manager dei power-up deve aumentare di uno il numero di vite. Le vite, però, sono gestite dal `LivesManager`. Quest'ultimo viene passato al metodo `updatePowerUp` come parametro: il manager dei power-up lo riceve solo quando ne ha bisogno e gli chiede di aggiungere una vita chiamando `addLife()`. Finita l'operazione, il manager dei power-up non tiene memoria del `LivesManager`, perché la comunicazione tra i due serve solo in quell'istante. Per avvisare anche il pannello laterale che una vita è stata appena guadagnata, il `LivesManager` mantiene un piccolo flag interno: il metodo `addLife()` lo setta, mentre il metodo `isLifeGained()` la legge e lo disattiva subito. In questo modo il pannello, consultando la bandiera ad ogni frame, riesce a mostrare l'icona del cuore una sola volta per ogni vita guadagnata e non a ripetizione.

➤ **Gestione della visibilità a schermo dei poteri**

Il giocatore deve sapere, in ogni momento, quali effetti sono attivi e per quanto tempo continueranno ad esserlo. Il pannello sinistro mantiene un array di posizioni occupate, una per effetto attivo, dove ogni posizione contiene il tipo di effetto, il numero di frame residui e il riferimento all'etichetta grafica che lo rappresenta. Ad ogni frame il controller, dopo aver aggiornato i timer del manager, comunica al pannello quali timer sono ancora attivi e quali sono scaduti, usando i metodi `addEffect` e `removeEffect`.

Il pannello pensa al resto: se il tipo di effetto è già presente aggiorna semplicemente il contatore residuo, altrimenti aggiunge una nuova icona; per gli effetti scaduti rimuove l'icona corrispondente e compatta l'array.

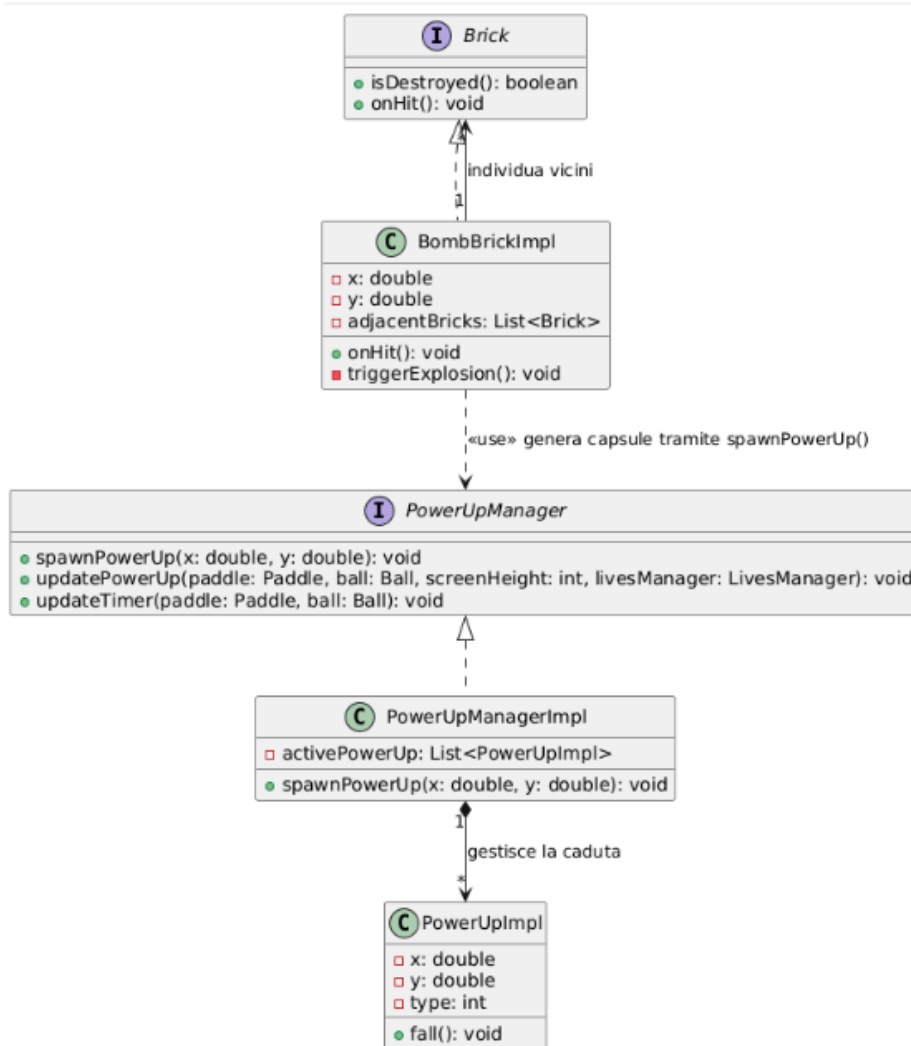


- **Aspetto: gestione dei blocchi esplosivi**

I blocchi esplosivi sono progettati per creare un'esplosione che distrugge tutti i blocchi distrutibili con un colpo di pallina al loro fianco (l'esplosione causerà 1 danno a tutti i blocchi adiacenti)

- **Soluzioni e considerazioni alternative:**

Si è scelto di implementare la logica dell'esplosione calcolando le coordinate dei blocchi vicini rispetto alla posizione del blocco bomba colpito. Quando il sistema rileva la collisione con il blocco esplosivo, non si limita a rimuovere solo quest'ultimo, ma esegue un controllo sulle quattro direzioni adiacenti (sopra, sotto, destra e sinistra). Se in una di queste posizioni è presente un blocco distrutibile, il gioco simula l'effetto di un colpo di pallina su di esso. Questo approccio centralizzato garantisce che la distruzione avvenga in un unico frame e che i blocchi bonus eventualmente coinvolti nell'esplosione attivino regolarmente la nascita di una capsula PowerUpImpl mantenendo la coerenza del gameplay.

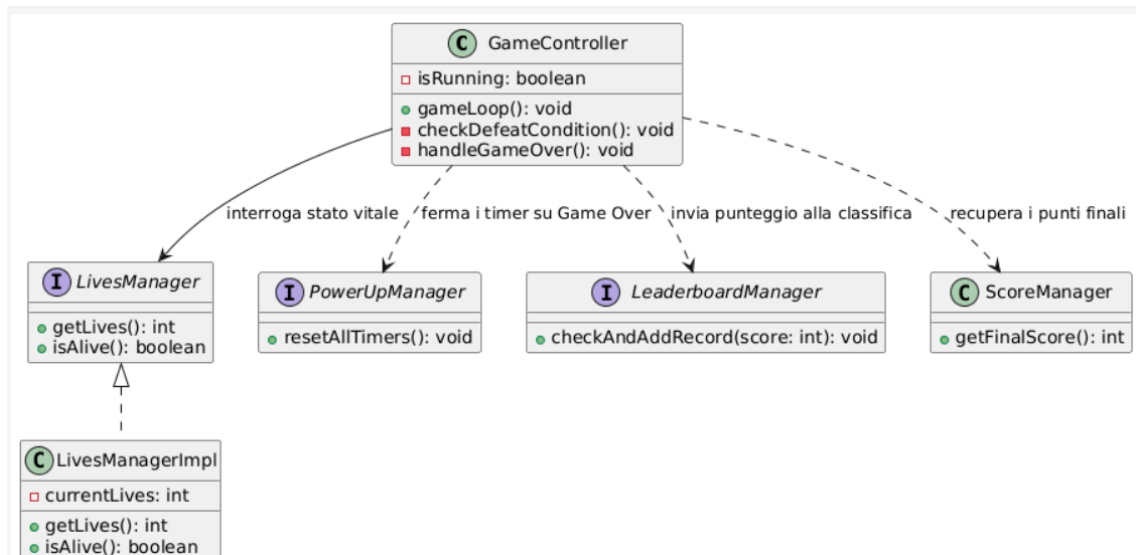


- **Aspetto: condizioni di fine gioco**

Il sistema deve monitorare costantemente lo stato della partita per decretare in tempo reale il momento esatto in cui il giocatore vince o perde.

- **Soluzioni e considerazioni alternative:**

La logica di controllo è stata centralizzata nel ciclo principale del Controller. Ad ogni frame viene valutata la condizione di sconfitta: viene determinata esclusivamente tramite il controllo delle vite, quando il giocatore perde tutte le vite a sua disposizione, il gioco termina immediatamente facendo visualizzare il punteggio effettuato.

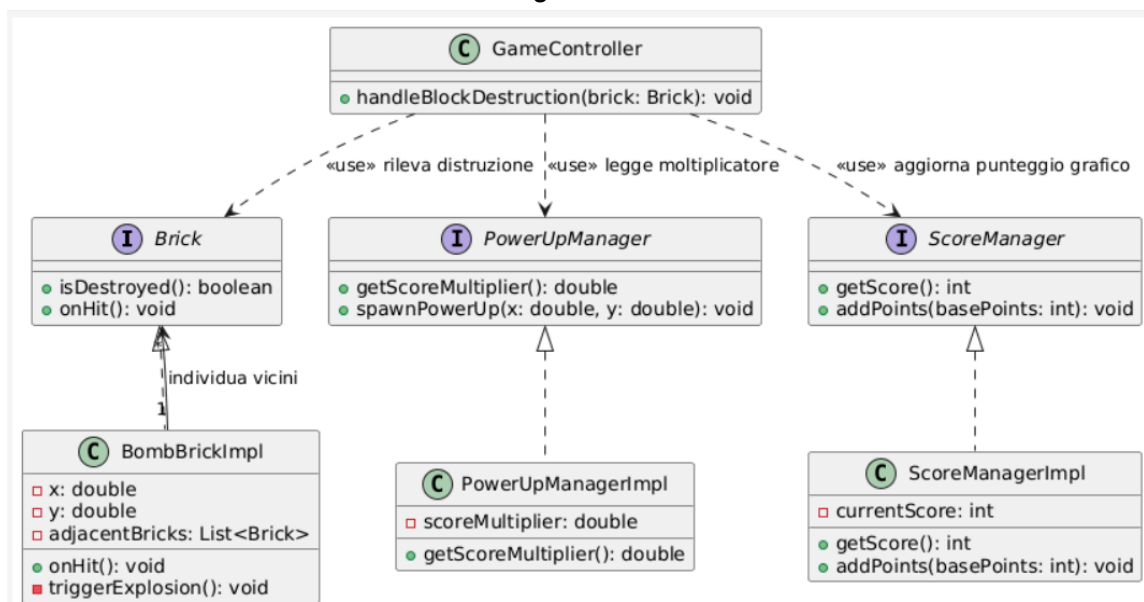


- **Aspetto: gestione del sistema di punteggio**

Il sistema deve calcolare, aggiornare e memorizzare il punteggio del giocatore in tempo reale durante la partita che si integri anche con i modificatori dovuti ai power up.

- **Soluzioni e considerazioni alternative**

Il calcolo del punteggio è stato centralizzato nel Model. Ogni blocco base distrutto ha un valore di partenza standard. Quando si verifica una collisione e il blocco viene rimosso, questo valore base viene moltiplicato per il coefficiente `scoreMultiplier` gestito dal **PowerUpManagerImpl**. Il moltiplicatore può assumere tre stati: 1.0 (stato base), 2.0 se è attivo il bonus Punti Doppi, 0.5 se è attivo il malus Punti Dimezzati. In caso di attivazione di un blocco esplosivo, i blocchi adiacenti distrutti dall'esplosione vengono calcolati nel medesimo frame: il sistema applica il moltiplicatore corrente a ciascun blocco abbattuto dalla reazione, garantendo un incremento immediato.



2.2.3 Cavina

Modellazione dell'entità Ball (Design by Contract)

Problema. La pallina è l'entità centrale del gioco: si muove ogni frame, rimbalza su pareti, paddle e mattoni, e viene persa quando cade oltre il bordo inferiore. Il problema di design principale consisteva nel definire il contratto dell'interfaccia Ball in modo che CollisionManager potesse applicare i rimbalzi senza dover accedere direttamente ai campi interni di velocità. Un'interfaccia che esponesse solo posizione e velocità scalare avrebbe costretto il gestore delle collisioni a leggere velocityX e velocityY, negarli e riscriverli dall'esterno - violando l'incapsulamento e distribuendo la logica di rimbalzo fuori dall'entità che la possiede.

Soluzione. L'interfaccia Ball espone metodi comportamentali espliciti: bounceX() e bounceY() incapsulano l'inversione della componente di velocità orizzontale e verticale, move() applica lo spostamento di un frame, isOutOfBounds(fieldHeight) centralizza il rilevamento della caduta, e setPosition(x, y) gestisce il riposizionamento al respawn. BallImpl mantiene le coordinate come double per garantire precisione sub-pixel nel movimento, e il costruttore valida che il raggio sia positivo lanciando IllegalArgumentException in caso contrario (design by contract). In questo modo CollisionManager interagisce con Ball esclusivamente attraverso il suo contratto pubblico, senza conoscere la rappresentazione interna. Lo schema UML dell'interfaccia Ball e della sua implementazione è riportato in Figura 2.2.

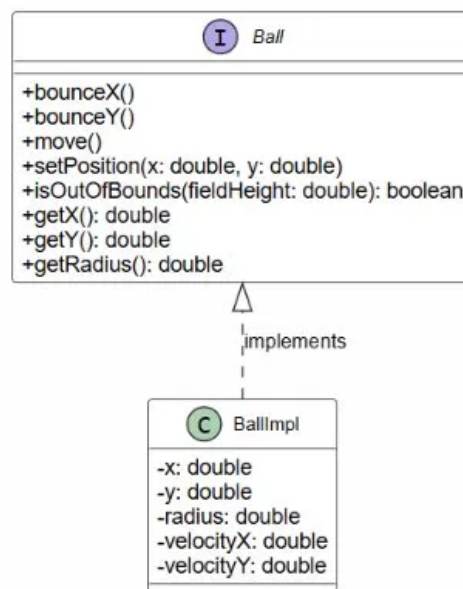


Figura 2.2: Schema UML dell'interfaccia Ball e della sua implementazione.

Cablaggio MVC (Factory Method + Command)

Problema. Creare e collegare tutti i componenti MVC (model, view, controller) senza introdurre accoppiamento circolare è un problema classico di bootstrap. Se GameController si costruisse le proprie dipendenze internamente, dovrebbe importare GameMapImpl - una classe concreta della view - creando una dipendenza diretta tra controller e implementazione grafica. Inoltre, la funzionalità "Play Again" richiedeva di ricreare l'intero sistema da zero senza riavviare il processo JVM, il che escludeva soluzioni basate su stato globale o singleton.

Soluzione. App è progettata come utility class (final, costruttore privato) con un unico factory method statico startGame() che istanzia in sequenza le entità del model, la view e il controller, passando a quest'ultimo solo le interfacce del model e il riferimento alla view. Il "Play Again" è implementato passando App::startGame come Runnable al controller e alla schermata di fine partita: quando l'utente sceglie di rigiocare, il callback ricrea l'intero sistema invocando nuovamente startGame(), garantendo un reset completo e pulito senza stato residuo. Il pattern Command (il Runnable come azione incapsulata) disaccoppia chi decide di riavviare (la GameOverView) da come il riavvio viene effettuato (App). La struttura del pattern è illustrata in Figura 2.3.

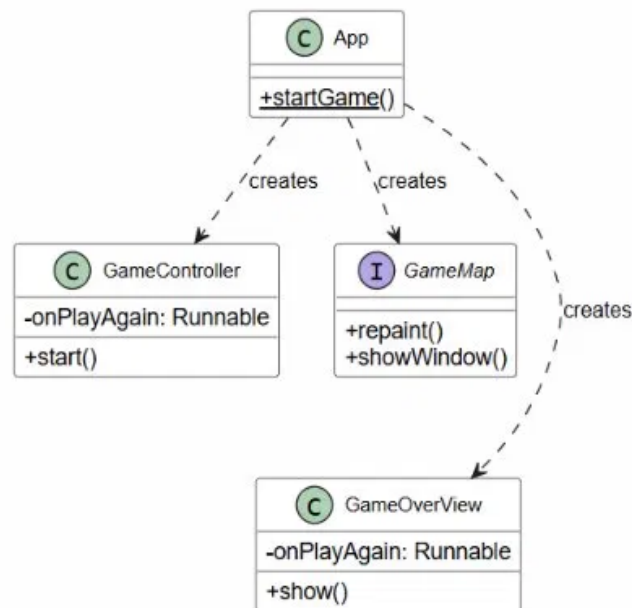


Figura 2.3: Schema UML del pattern Factory Method e Command nel cablaggio MVC.

Gestione delle transizioni di schermata (Command)

Problema. Il ciclo di vita dell'applicazione attraversa tre schermate distinte (menu → gioco → game over) e deve garantire che ogni transizione avvenga senza finestre residue e senza stato del model contaminato dalla sessione precedente.

Soluzione. Ogni schermata (MenuView, GameOverView) riceve nel costruttore uno o più callback Runnable che ne definiscono le azioni possibili. Al momento della transizione, la schermata chiama frame.dispose() per distruggere la propria finestra e poi invoca il callback appropriato - onPlayAgain.run() o System.exit(0). La scelta di distruggere e ricreare la finestra ad ogni transizione, invece di usare un CardLayout che swappa pannelli sulla stessa finestra, garantisce che tutto lo stato grafico e del model venga ricreato da zero: non è possibile che variabili di istanza della sessione precedente sopravvivano alla transizione. L'invocazione del callback avviene tramite SwingUtilities.invokeLater() per assicurare che la transizione avvenga sul thread EDT, evitando race condition tra la chiusura della finestra corrente e la creazione della nuova. Lo schema delle transizioni è riportato in Figura 2.4.

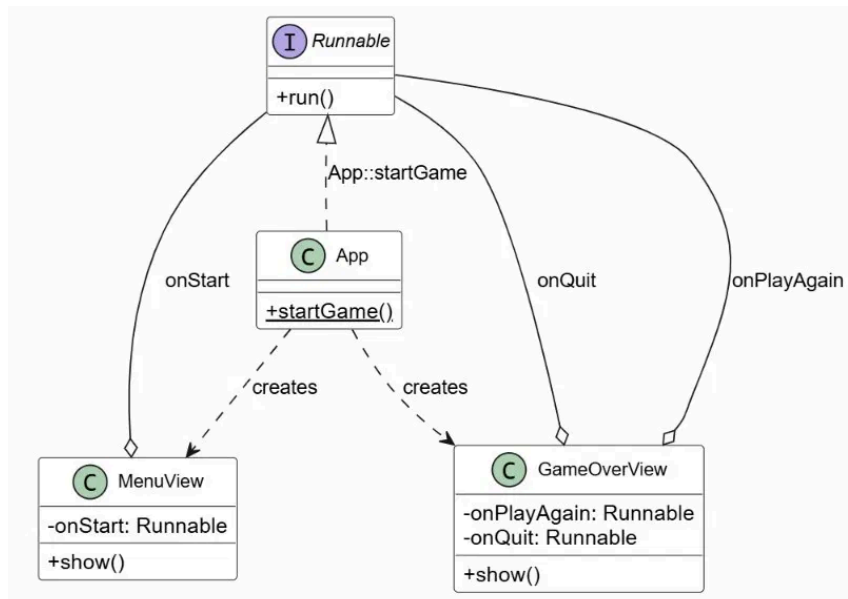


Figura 2.4: Schema UML del pattern Command nella transizione di schermata.

2.2.4 Frega

Gestione del paddle (incapsulamento della geometria e responsività)

Problema: Il paddle deve muoversi orizzontalmente restando sempre dentro il campo di gioco, deve poter cambiare dimensione per effetto dei power up ma entro limiti sensati, e deve adattare dimensione e posizione al ridimensionamento della finestra (requisito di responsività). Se la logica dei confini, dei limiti di dimensione e del riscaldamento fosse affidata al controller o alla view, finirebbe sparpagliata in componenti che non dovrebbero conoscere i dettagli interni del paddle, e gli invarianti dell'entità — non uscire mai dal campo, non rimpicciolirsi o ingrandirsi oltre misura — potrebbero essere violati dall'esterno. Inoltre, fissare le dimensioni in pixel assoluti renderebbe impossibile rispettare la responsività su schermi di risoluzione diversa.

Soluzione: Tutti gli invarianti geometrici sono incapsulati nell'entità stessa (information hiding e Single Responsibility). Il paddle si sposta tramite `moveLeft()` e `moveRight()` e custodisce da sé il confine del campo con `clamp(screenWidth)`: il controller si limita a dire "muovi" e "rientra nei limiti", senza dover calcolare i bordi. I cambi di dimensione dovuti ai power up (`paddleLarge()` e `paddleShort()`) sono vincolati tra `MIN_WIDTH` e `MAX_WIDTH` all'interno del paddle, così nessun codice esterno può portarlo fuori scala. Per la responsività, `updateDimensions()` esprime la larghezza e la posizione verticale come proporzioni del pannello (una frazione della larghezza e una dell'altezza) anziché in pixel fissi: al ridimensionamento il paddle si riscalda da solo in modo coerente. Infine, il paddle si espone al resto del sistema tramite l'astrazione di sola lettura `Collidable`, così la view può disegnarlo ma non spostarlo, e partecipa in modo uniforme al rilevamento delle collisioni.

Lo schema è riportato nella figura 2.5

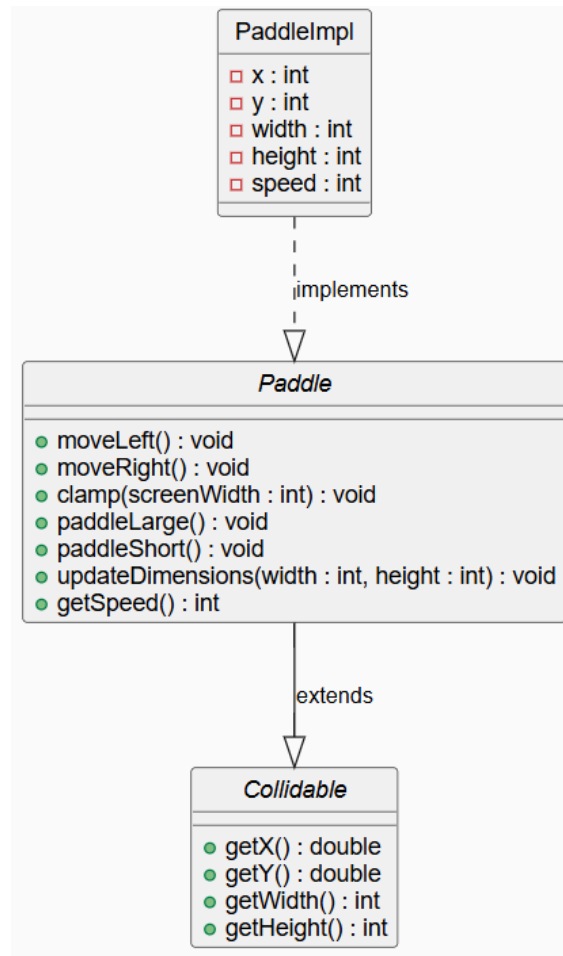


Fig 2.5: Schema UML del paddle

Gestione delle collisione tra gli oggetti (pattern strategy)

Problema: Gestire le collisioni tra la pallina e oggetti eterogenei (paddle, bordi del campo, mattoncini di tipi diversi) è delicato: se ogni entità gestisse le proprie collisioni, Ball, Paddle e Brick dovrebbero conoscersi a vicenda, disperdendo la logica e introducendo accoppiamento tra le entità del model. Inoltre il rilevamento geometrico di una collisione e la sua risoluzione (rimbalzo, punteggio, suoni) sono concetti distinti che, se mescolati, legherebbero la fisica alle regole di gioco. C'è poi il problema di comunicare al resto del sistema cosa è successo in un singolo frame — quale mattoncino è stato colpito, se è stato toccato un bordo o il paddle — per attivare suoni ed effetti, senza che il gestore delle collisioni dipenda dalla view o dall'audio. Infine, il rimbalzo sul paddle non può essere una semplice riflessione geometrica: il giocatore deve poter controllare la traiettoria in base al punto in cui la pallina colpisce.

Soluzione: Tutta la logica è centralizzata in `CollisionManagerImpl` tramite un unico metodo `handleCollisions(...)`, così le entità del model (Ball, Paddle, Brick) restano prive di logica di collisione. Il rilevamento geometrico è delegato all'interfaccia `CollisionDetector` (pattern Strategy), che opera sull'astrazione di sola lettura `Collidable` (sola posizione e dimensioni): in questo modo il controllo funziona su qualunque oggetto posizionato senza conoscerne il tipo concreto, ed è facilmente sostituibile e mockabile nei test. La comunicazione con il controller avviene tramite flag "a consumo singolo" (`isPadHit`, `isBorderHit`, `typeOfBlockHit`) che si

azzerano alla lettura: il controller interroga una volta per frame cosa è stato colpito e attiva di conseguenza suoni ed effetti, senza che il manager debba conoscere SoundManager o la view, invertendo così la direzione della dipendenza. Il rimbalzo sul paddle mappa l'offset del punto d'impatto rispetto al centro su un angolo di lancio (0° al centro, fino a $\pm 60^\circ$ ai bordi) conservando il modulo della velocità, ottenendo il controllo "arcade" tipico del gioco anziché una riflessione fisica. Il punteggio, infine, è calcolato in un unico punto applicando il moltiplicatore fornito dal PowerUpManager. Lo schema delle collisioni è riportato nella figura 2.6

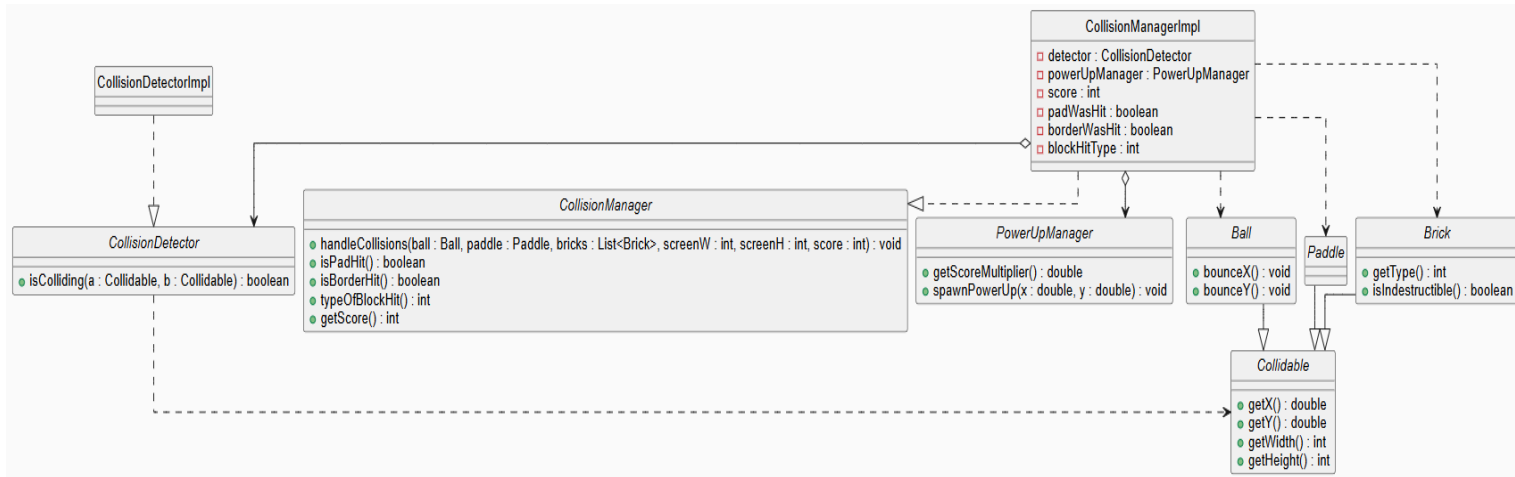


Fig 2.6: Schema UML delle collisioni

3. Sviluppo

3.1 Testing Automatizzato

Al fine di garantire che l'evoluzione del codice non introduca problemi sulle componenti logiche e fisiche dell'applicazione, è stata sviluppata una suite di test d'unità e di integrazione automatico utilizzando il framework JUnit 5.

Suite di Test: TestBall:

- La suite si occupa di verificare il contratto comportamentale dell'entità Ball attraverso 28 test unitari.
 - **Costruttore e validazione:** Verifica che un raggio pari a zero o negativo lanci `IllegalArgumentException` (design by contract), e che un raggio positivo non produca eccezioni.
 - **Movimento:** Controlla che `move()` aggiorni correttamente la posizione sommando la velocità corrente, con accumulo su chiamate multiple e gestione di velocità negative.
 - **Rimbalzi:** Verifica che `bounceX()` e `bounceY()` invertano rispettivamente la componente orizzontale e verticale della velocità, lasciando invariata la componente opposta. Un doppio bounce ripristina la velocità originale.
 - **Riposizionamento:** Valida che `setPosition()` aggiorni le coordinate senza alterare la velocità.
 - **Uscita dal campo:** Assicura che `isOutOfBounds()` restituisca true solo quando il bordo inferiore della pallina supera strettamente la soglia, e false quando la tocca esattamente.
 - **Ridimensionamento:** Testa entrambi i rami di `updateDimensions()`: il clamp della pallina in volo entro i bordi del pannello, e il ricentrimento della pallina ferma sul paddle prima del lancio.

Suite di Test: TestBrick:

- **TestBrick:** (Validazione dell'entità `BrickImpl`) Questa suite si occupa di verificare il corretto ciclo di vita, i cambiamenti di stato e la fisica del singolo blocco.
 - **Ciclo di vita e resistenza:** Verifica che i blocchi normali (1 vita) e resistenti (2 vite) rispondano coerentemente alle collisioni
 - **Indistruttibilità:** Assicura che un blocco marcato come indistruttibile (tipo 3) non riduca la propria vita e non risulti mai distrutto
 - **Blocchi speciali:** Valida l'inizializzazione e il comportamento dei blocchi speciali (tipo 4 e 5), verificando che possiedano una sola vita e che rispondano correttamente alla distruzione.
 - **Geometria e Movimento:** Testa il corretto aggiornamento delle coordinate spaziali a seguito dell'invocazione di `moveDown(amount)` e valida il funzionamento dei metodi accessori (getter/setter) per il posizionamento e il dimensionamento dei blocchi.

Suite di Test: LevelManagerImplTest:

- **LevelManagerImplTest:** Questa suite si occupa nel controllare il corretto movimento e la creazione della griglia di blocchi all'interno del gioco

- **Inizializzazione e reset:** Verifica che il metodo `reset()` configuri correttamente lo stato iniziale, azzeri la velocità di scorrimento al valore di base, ripristini i contatori e generi la quantità corretta di righe iniziali
- **Immutabilità della vista:** Valida che la lista restituita da `getActiveBricks()` sia effettivamente una vista non modificabile
- **Fisica dello scorrimento e accelerazione:** Controlla che l'invocazione del metodo `update(deltaTime)` trasli in modo corretto i blocchi verso il basso, elimini i mattoncini che superano il confine inferiore dello schermo, generi nuove righe fuori dall'area visiva dopo un intervallo di tempo e aumenti progressivamente la velocità di discesa.
- **Gestione delle rimozioni:** Verifica la corretta eliminazione di singoli blocchi (tramite `removeBrick`) e testa la pulizia collettiva dei blocchi distrutti (tramite `removeDestroyedBricks`).
- **Rilevamento della soglia:** Assicura che il sistema rilevi correttamente quando il bordo inferiore di almeno un mattoncino attraversa la coordinata di soglia del paddle.
- **Ridimensionamento dinamico (Resize):** Valida il comportamento di `updateDimensions(width, height)`. Testa l'immunità a input non validi (dimensioni negative o nulle) e verifica che, a seguito di ridimensionamenti successivi, le coordinate (X, Y) e le dimensioni (larghezza, altezza) di tutti i blocchi attivi vengano riscalate rispetto al fattore di scala calcolato.

Suite di Test: **TestPowerUp:**

- **TestPowerUp:** Questa suite si occupa di verificare le seguenti funzionalità del modulo di gestione dei bonus/malus:
 - **Stato Iniziale e Prevenzione Regressioni:** Verifica che all'avvio del gioco o all'inizializzazione del manager non vi siano modificatori attivi (il gioco non deve risultare congelato), il moltiplicatore dei punti sia impostato al valore base (1.0) e la lista delle capsule presenti a schermo sia inizialmente vuota. Questo assicura che il reset del sistema riporti sempre il modello a uno stato coerente.
 - **Cinematica e Confini delle Capsule:** Controlla le proprietà di base della singola capsula (tipo e coordinate). Valida il metodo `fall()`, assicurandosi che ad ogni frame l'oggetto si sposti verso il basso incrementando la coordinata Y della corretta velocità di caduta senza alterare la X. Verifica inoltre che il rilevamento del superamento dei confini dello schermo (`isOutOfBounds`) funzioni correttamente sia all'interno che all'esterno dei limiti, compreso il caso di test sul bordo esatto.
 - **Generazione Automatica (Spawn):** Assicura che l'invocazione di `spawnPowerUp` inserisca correttamente una nuova capsula all'interno della lista dei poteri attivi e che il tipo di effetto assegnato casualmente rientri strettamente nel range di ID validi previsti dal gioco (da 1 a 7).
 - **Raccolta ed Attivazione Effetti:** Testa singolarmente l'impatto di ciascun power-up quando viene intercettato dal paddle. Verifica che i timer di durata vengano impostati al numero di frame prefissato per gli effetti persistenti (paddle corto/lungo, punti doppi/dimezzati, congelamento blocchi, pallina

- veloce). Per i bonus istantanei come la vita extra, controlla l'incremento effettivo nel gestore delle vite e l'attivazione del rispettivo flag di notifica.
- **Sovrascrittura e Conflitti:** Valida la logica di mutua esclusione tra gli effetti contrari, accertandosi che la raccolta di un paddle lungo annulli immediatamente il timer del paddle corto (e viceversa) e che il depotenziamento dei punti dimezzati azzeri il bonus dei punti doppi. Verifica inoltre che la raccolta consecutiva dello stesso identico bonus provochi il reset del timer alla durata massima iniziale.
 - **Scadenza e Ripristino dei Valori:** Controlla che ad ogni frame il metodo `updateTimer` decrementi regolarmente la durata dei poteri attivi. Assicura che allo scadere del tempo i modificatori vengano revocati in automatico, ripristinando i valori di base del gioco (come il moltiplicatore del punteggio o la velocità standard della pallina).
 - **Pulizia delle Capsule:** Verifica che le capsule non intercettate dal giocatore vengano rimosse dalla lista attiva una volta superata la coordinata del fondo dello schermo, mentre quelle ancora in gioco continuino la loro discesa in assenza di collisioni.
- **TestPaddle:** La suite verifica il contratto comportamentale dell'entità Paddle attraverso 9 test unitari.
 - **Movimento:** controlla che `moveLeft()` e `moveRight()` spostino la coordinata orizzontale della velocità corrente, rispettivamente decrementandola ed incrementandola.
 - **Contenimento nei bordi:** verifica che `clamp()` impedisca al paddle di uscire dal campo, riportandolo a 0 quando oltrepassa il bordo sinistro e a (larghezza dello schermo – larghezza del paddle) quando oltrepassa quello destro.
 - **Bonus e malus:** valida che `paddleLarge()` e `paddleShort()` modifichino la larghezza di una quantità fissa, rispettando però i limiti, ossia senza allargarsi oltre il massimo né restringersi sotto il minimo consentito.
 - **Ridimensionamento responsive:** testa `updateDimensions()`, che ricalcola larghezza e posizione verticale come proporzioni del pannello, mantenendo il paddle entro i confini.
 - **TestCollisions:** La suite verifica la logica del `CollisionManager` attraverso 11 test unitari, isolando l'unità tramite test double (classi Fake) per la palla, i mattoncini, il rilevatore di collisioni e il gestore dei power up.
 - **Eventi di collisione (consumo singolo):** verifica che il colpo al paddle, al bordo e a un blocco vengano segnalati una sola volta, ossia che `isPadHit()`, `isBorderHit()` e `typeOfBlockHit()` restituiscano l'esito e si azzerino alla lettura, controllando inoltre che il tipo del blocco colpito venga salvato correttamente.
 - **Calcolo del punteggio:** controlla che `getScore()` restituisca il punteggio iniziale, che un blocco distrutto aggiunga 300 punti, uno colpito ma non distrutto ne aggiunga 150, e che un blocco indistruttibile non alteri il punteggio.

- **Moltiplicatore dei power up:** valida che il moltiplicatore fornito dal PowerUpManager venga applicato correttamente al punteggio (ad esempio, punti raddoppiati).
- **Fisica del rimbalzo sul paddle:** verifica che un colpo al centro produca una traiettoria dritta verso l'alto (angolo nullo), un colpo ai bordi una deviazione di $\pm 60^\circ$ dalla verticale, e che in ogni caso il modulo della velocità si conservi.

Injection tramite reflection

Per isolare i test di integrazione dalle componenti randomiche di generazione automatica e testare le transizioni di stato in scenari specifici, è stata utilizzata la Reflection di Java. All'interno del metodo ausiliario injectBricks(List<Brick> bricks) della classe di test, viene estratto a runtime il campo privato activeBricks di LevelManagerImpl per mezzo dell'API java.lang.reflect.Field. Questa tecnica consente di superare il modificatore di accesso private e di svuotare o sovrascrivere direttamente la collezione interna dei mattoncini attivi con una lista preconfigurata di test, così facendo poter testare tutti gli scenari senza dover dipendere dalla generazione casuale.

3.2 Note di sviluppo

Montefiori

- Utilizzo di riflessione (java reflection API):

API di reflection (java.lang.reflect.Field) per l'accesso dinamico ai membri di una classe, è stata utilizzata all'interno della suite di test in LevelManagerImplTest.java (metodo injectBricks) per accedere a runtime al campo privato activeBricks della classe sotto test, renderlo accessibile ed effettuare l'injection controllata di una lista fittizia (mock), permettendo di testare le logiche in modo completo.

Permalink: <https://github.com/fabiocavina22/OOP25-oop-brk25/blob/c7dbcb4c4aa7293506b3afb21aae986d599f444c/src/test/java/it/unibo/breakout/TestLevel.java#L28-L35>

- Utilizzo delle espressioni lambda e riferimenti a metodi (method references):

Sfruttate in LevelManagerImpl.java (metodo update) per eseguire la pulizia della lista activeBricks, ad esempio tramite il metodo removeIf con il riferimento a metodo Brick::isDestroyed. e usate molto spesso, il permalink sottostante è un esempio

Permalink: <https://github.com/fabiocavina22/OOP25-oop-brk25/blob/c7dbcb4c4aa7293506b3afb21aae986d599f444c/src/main/java/it/unibo/breakout/model/impl/LevelManagerImpl.java#L94-L95>

- Fisica tempo-dipendente dello scorrimento (Delta Time):

Sfruttata all'interno del metodo update per aggiornare lo spostamento spaziale di ogni blocco moltiplicando la velocità costante (scrollSpeed) per il tempo reale trascorso dall'ultimo frame (deltaTime).

Permalink: <https://github.com/fabiocavina22/OOP25-oop-brk25/blob/c7dbcb4c4aa7293506b3afb21aae986d599f444c/src/main/java/it/unibo/breakout/model/impl/LevelManagerImpl.java#L89-L92>

- API Stream e Collezionatori (Collectors.groupingBy):

Nel metodo removeIndestructibleFromClearedRows di LevelManagerImpl, per implementare la pulizia automatica delle righe è stata sfruttata la programmazione funzionale dall'API

Stream, gli elementi vengono convertiti in Stream e raggruppati in una mappa tramite l'id della loro riga, per poi guardare se dentro ciascuna lista ci siano ancora blocchi distrutibili per poi così venire rimossi dalla memoria con l'operazione di remove

Permalink:<https://github.com/fabiocavina22/OOP25-oop-brk25/blob/c7dbcb4c4aa7293506b3afb21aae986d599f444c/src/main/java/it/unibo/breakout/model/impl/LevelManagerImpl.java#L244-L253>

Cavina

- Method reference come Runnable (App::startGame):
<https://github.com/fabiocavina22/OOP25-oop-brk25/blob/1cd098c4fa4f1c6c867f7f834d465596e1d6d1ed/src/main/java/it/unibo/breakout/App.java#L41>
- SwingUtilities.invokeLater():
<https://github.com/fabiocavina22/OOP25-oop-brk25/blob/1cd098c4fa4f1c6c867f7f834d465596e1d6d1ed/src/main/java/it/unibo/breakout/controller/impl/GameController.java#L133-L136>
- MouseAdapter anonimo (override parziale di MouseListener):
<https://github.com/fabiocavina22/OOP25-oop-brk25/blob/1cd098c4fa4f1c6c867f7f834d465596e1d6d1ed/src/main/java/it/unibo/breakout/view/impl/MenuView.java#L98-L108>

Di Domenico

- Utilizzo della programmazione funzionale tramite l'API Stream per la gestione e l'elaborazione dei dati della classifica. I punteggi dei giocatori vengono convertiti in stream, filtrati, ordinati in ordine decrescente tramite comparatori funzionali e infine collezionati:
<https://github.com/fabiocavina22/OOP25-oop-brk25/blob/c7dbcb4c4aa7293506b3afb21aae986d599f444c/src/main/java/it/unibo/breakout/model/impl/LevelManagerImpl.java#L244-L253>
- Sfruttate all'interno del PowerUpManagerImpl e dei componenti di gestione della Leaderboard per manipolare le collezioni di dati in modo dichiarativo. Ad esempio PowerUpImpl::isOutOfBounds all'interno di metodi come removelf per ripulire la memoria dalle capsule scadute o oltrepassate, riducendo la verbosità del codice ed evitando l'uso di iteratori imperativi manuali:
<https://github.com/fabiocavina22/OOP25-oop-brk25/blob/c7dbcb4c4aa7293506b3afb21aae986d599f444c/src/main/java/it/unibo/breakout/model/impl/LevelManagerImpl.java#L94-L95>

Frega

- Uso di javax.sound.sampled (libreria JDK non spiegata a lezione)
<https://github.com/fabiocavina22/OOP25-oop-brk25/blob/main/src/main/java/it/unibo/breakout/view/impl/SoundManagerImpl.java#L1-L10>
- Lambda expression come LineListener (listener anonimo scritto come lambda)
<https://github.com/fabiocavina22/OOP25-oop-brk25/blob/main/src/main/java/it/unibo/breakout/view/impl/SoundManagerImpl.java#L40-L49>

4. Commenti finali

4.1 Autovalutazione e lavori futuri

Montefiori

All'interno del gruppo mi sono occupato del design e dello sviluppo del sottosistema dedicato alla gestione dinamica dei livelli e alla modellazione dei mattoni. Nello specifico, ho progettato e implementato l'interfaccia LevelManager con la classe concreta LevelManagerImpl, l'astrazione e l'implementazione dei mattoni (Brick e BrickImpl) e la classe di utilità BrickFactory per la generazione centralizzata delle istanze.

Un grande punto è il modo in cui vengono creati i blocchi. Ho utilizzato una factory che sarebbe BrickFactory che si occupa di generare tutti i blocchi. In questo modo, chi gestisce il livello non deve conoscere i dettagli di ogni singolo mattone. Se in futuro si volesse aggiungere un nuovo tipo di blocco, basterebbe modificare solo la fabbrica, senza rischiare di rompere il resto del codice.

Inoltre un altro vantaggio del lavoro è lo scorrimento del gioco in modo abbastanza fluido. Il sistema che calcola i movimenti in base al tempo che passa e non alle prestazioni del computer, quindi il gioco va alla stessa velocità sia su computer lenti che su quelli con prestazioni migliori. Inoltre, la parte grafica è separata dalle regole logiche, rendendo il codice più pulito.

Il punto debole, invece, si nota quando si allarga o si stringe la finestra di gioco molto velocemente: la grafica fa fatica a stare dietro ai calcoli del modello, creando uno sfarfallio visivo. Inoltre, tutti i suoni e le immagini vengono caricati tutti insieme all'avvio del gioco, cosa che potrebbe rallentare la partenza se decidessimo di aggiungere molte altre risorse in futuro.

Un altro difetto è che molte regole del gioco sono scritte direttamente all'interno del codice in modo fisso. Ad esempio, la velocità di scorrimento del livello o la frequenza con cui compaiono i mattoncini speciali non si possono regolare dall'esterno. Per cambiarle bisogna per forza modificare il codice e ricompilare il programma.

Riguardo il futuro di questo lavoro mi piacerebbe migliorare aspetti come i disegni predefiniti, cioè invece di far comparire righe di mattoni del tutto casuali, mi piacerebbe creare dei "disegni" di blocchi già pronti da far apparire a sorpresa, rendendo il livello infinito meno ripetitivo e più divertente ed anche fare in modo che le immagini e i suoni vengano caricati in sottofondo mentre il gioco si avvia, evitando qualsiasi attesa iniziale.

Di domenico

All'interno del gruppo mi sono occupato dello sviluppo del sottosistema dedicato alla gestione logica e grafica dei Power-Up e del sistema di persistenza e visualizzazione della Leaderboard (classifica dei punteggi).

Nello specifico, ho progettato e implementato l'interfaccia PowerUpManager con la rispettiva classe concreta PowerUpManagerImpl per il ciclo di vita dei bonus/malus, l'interfaccia di isolamento PowerUpView.

Un grande punto di forza è l'utilizzo dell'interfaccia di sola lettura PowerUpView. Grazie a questo accorgimento architetturale, la View può interrogare il manager per sapere quali capsule disegnare a schermo, ma non ha l'accesso ai metodi di controllo (come lo spawn o la modifica dei timer).

Questo garantisce un incapsulamento solido e azzerà il rischio che la grafica possa alterare accidentalmente lo stato del gioco. Il sistema dei timer è robusto ed efficiente: la gestione dei conflitti (come il raccogliere un effetto "Paddle Lungo" che annulla immediatamente l'effetto attivo "Paddle Corto") e il reset dei frame residui in caso di bonus consecutivi funzionano in modo del tutto automatico e senza regressioni. Anche la classifica si integra perfettamente con il flusso di fine partita.

Il principale punto debole risiede nel sistema di persistenza della classifica: i punteggi della Leaderboard vengono salvati localmente su un file di testo semplice.

Questo approccio, sebbene economico e facile da implementare, espone i dati al rischio di manomissione esterna da parte dell'utente, che potrebbe modificare manualmente il file per alterare i record. Un altro difetto è legato alla rigidità dei parametri dei Power-Up, come la durata fissa dei timer impostata a 500 frame o il valore dei modificatori di punteggio (2.0 e 0.5). Queste proprietà sono scritte direttamente all'interno del codice sotto forma di costanti; di conseguenza, non è possibile bilanciare il gameplay o variare la durata dei poteri a seconda del livello di difficoltà senza dover modificare il codice sorgente e ricompilare l'intero programma.

Se dovessi portare avanti il progetto, l'idea principale sarebbe quella di rendere la Leaderboard più sicura e moderna, ad esempio implementando una cifratura per il file di salvataggio locale o, idealmente, collegando il gioco a un database online tramite API dedicate, così da permettere una classifica globale condivisa tra più giocatori.

Dal punto di vista dei Power-Up, mi piacerebbe introdurre una maggiore varietà di effetti (come una barriera protettiva sul fondo o palline sdoppiate) e rendere il sistema più dinamico.

Cavina

Mi sono occupato dell'entità centrale del gioco (Ball), dell'interfaccia utente di base (menu principale e schermata di fine partita) e del cablaggio dei componenti MVC tramite il factory method `App::startGame`.

Ritengo che il punto di forza del mio lavoro sia la modellazione dell'interfaccia Ball: esporre metodi comportamentali espliciti come `bounceX()` e `bounceY()` invece di delegare la fisica all'esterno ha mantenuto l'incapsulamento e semplificato il lavoro del CollisionManager. Sono soddisfatto anche della soluzione adottata per le transizioni di schermata tramite callback Runnable, che garantisce un reset completo dello stato a ogni nuova partita senza riavviare la JVM.

In futuro mi piacerebbe aggiungere il supporto al mouse come alternativa alla tastiera per muovere il paddle, e migliorare la schermata di fine partita con statistiche più dettagliate sulla partita appena conclusa.

Frega

All'interno del gruppo mi sono occupato principalmente della gestione delle collisioni (rilevamento, rimbalzi e distruzione dei mattoni), del pannello centrale di gioco (MainPanel), del pannello laterale con HUD e indicatori dei tasti (LeftPanel), della Paddle e del sistema audio (SoundManagerImpl).

Il punto di forza del mio lavoro è la separazione netta tra rilevamento e gestione delle collisioni: CollisionDetectorImpl si occupa esclusivamente di determinare se due oggetti si sovrappongono, mentre CollisionManagerImpl gestisce tutta la logica conseguente (rimbalzi, punteggi, esplosioni a catena, rilascio di power-up). Questa divisione di responsabilità rende il codice facile da mantenere e testare. Sono soddisfatto anche della fisica del rimbalzo sulla paddle, che calcola dinamicamente l'angolo in funzione della posizione di impatto, rendendo il gioco più reattivo e strategico. Per quanto riguarda il comparto audio, ho gestito la chiusura automatica delle risorse tramite LineListener, evitando memory leak.

Come punto di debolezza, riconosco che CollisionManagerImpl accumula diverse responsabilità (punteggio, esplosioni, power-up, rimbalzi): in retrospettiva avrei potuto suddividerla ulteriormente, ad esempio delegando la logica dei mattoni esplosivi a un gestore separato.

Come sviluppo futuro, sarebbe interessante introdurre un sistema di collisioni più preciso basato sulla geometria reale degli oggetti, rendere la finestra ridimensionabile e rendere il sistema degli effetti visivi nella HUD configurabile tramite un registro esterno anziché hardcoded nella classe.

A. Guida utente

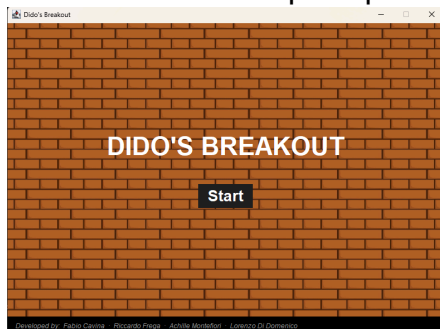
All'avvio dell'applicazione viene visualizzato il menu principale: premendo il pulsante *Start*¹ ha inizio la partita.

Durante il gioco, i tasti **A** e **D**² (o le frecce direzionali sinistra e destra) consentono di spostare il paddle orizzontalmente. Per mettere in pausa la partita è sufficiente premere il tasto **S**; per riprendere, premere nuovamente **S**.

Quando la pallina cade oltre il bordo inferiore il giocatore perde una vita: la pallina viene riposizionata al centro del paddle. È possibile scegliere la direzione di partenza spostando il paddle con **A** e **D**, quindi premere **W**³ per lanciare la pallina e riprendere la partita.

La partita termina quando le vite si esauriscono o quando una riga di mattoni raggiunge il paddle. Viene mostrata la schermata di fine partita con il punteggio ottenuto: se il risultato rientra tra i migliori dieci, il sistema chiede al giocatore di inserire tre caratteri⁴ (secondo la convenzione arcade). Confermando con **OK**, il proprio nome e il punteggio vengono registrati nella classifica persistente dei migliori risultati. Dalla stessa schermata è possibile scegliere *Play Again* per avviare una nuova partita o *Exit*⁵ per uscire dall'applicazione.

1: Interfaccia menù principale



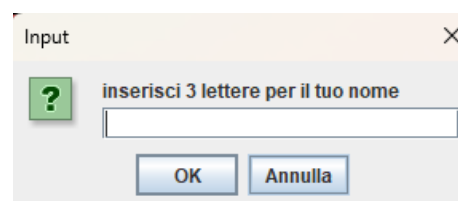
2: Tasti A e D su schermo del gioco



3: Tasto W su schermo del gioco



4: Input dove inserire le 3 lettere



5: Schermata play again o exit

